

DOSSIER PROFESSIONNEL (DP)

Nom de naissance ▶ AL OMRAN

Nom d'usage ▶ AL OMRAN

Prénom ▶ Abduljaber

Adresse ▶ 2 rue pierre baisle
63300 Thiers

Titre professionnel visé

Concepteur Développeur d'Applications (CDA)

MODALITÉ D'ACCÈS :

- Parcours de formation
☒
- ☐ Validation des Acquis de l'Expérience (VAE)

--

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel.

Ce titre est délivré par le Ministère chargé de l'emploi.

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen.**

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle.
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels

du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;

- ▶ un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- ▶ une déclaration sur l'honneur à compléter et à signer ;
- ▶ des documents illustrant la pratique professionnelle du candidat (facultatif)
- ▶ des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.



<http://travail-emploi.gouv.fr/titres-professionnels>

Sommaire

Exemples de pratique professionnelle

Développer une application sécurisée	p.	8
▶ Installer et configurer son environnement de travail en fonction du projet	p.	8
▶ Développer des interfaces utilisateur	p.	10
▶ Développer des composants métier	p.	12
▶ Contribuer à la gestion d'un projet informatique	p.	14
Concevoir et développer une application sécurisée organisée en couches	p.	16
▶ Analyser les besoins et maquetter une application	p.	16
▶ Définir l'architecture logicielle d'une application	p.	18
▶ Concevoir et mettre en place une base de données relationnelle	p.	20
▶ Développer des composants d'accès aux données SQL et NoSQL	p.	22
Préparer le déploiement d'une application sécurisée	p.	24
▶ Préparer et exécuter les plans de tests d'une application	p.	24
▶ Préparer et documenter le déploiement d'une application	p.	26
▶ Contribuer à la mise en production dans une démarche DevOps	p.	28
Titres, diplômes, CQP, attestations de formation <i>(facultatif)</i>	p.	30

Déclaration sur l'honneur

p
.

31

Documents illustrant la pratique professionnelle *(facultatif)*

p
.

32

Annexes *(Si le RC le prévoit)*

p
.

33

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer une application sécurisée

Exemple n°1 ▶ Installer et configurer son environnement de travail – projet PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Au lancement de mon projet chef-d'œuvre PromptVerse, une marketplace d'achat et de vente de prompts pour les IA génératives, j'ai mis en place l'environnement de développement complet avant d'écrire la première fonctionnalité. La stack était imposée par le brief : React TypeScript, NestJS, TypeORM et MySQL.

- Création du dépôt Git organisé en monorepo : apps/api pour l'API NestJS, apps/client pour l'application React créée avec Vite, docs/ pour la conception et openspec/ pour les spécifications.
- Initialisation du projet NestJS avec la CLI et configuration des scripts npm : lancement en développement, build, lint, tests, génération et exécution des migrations TypeORM.
- Écriture d'un fichier docker-compose.yml qui démarre en une commande tous les services nécessaires : MySQL 8.4, Redis pour le cache, MinIO (stockage compatible S3) avec un conteneur d'initialisation du bucket, Mailpit pour intercepter les emails de vérification et RedisInsight pour inspecter le cache.
- Gestion de la configuration par variables d'environnement, validées au démarrage de l'API : l'application refuse de démarrer si une variable obligatoire est absente ou invalide.
- Mise en place d'ESLint et de Prettier sur les deux applications et d'un Makefile pour les commandes courantes (démarrage des conteneurs, consultation des logs).
- Initialisation d'OpenSpec pour une démarche guidée par les spécifications : chaque module (authentification, utilisateurs, prompts, catégories et outils IA, paiement, Docker et CI/CD) fait l'objet d'une proposition versionnée.

Grâce à cet environnement, le projet s'installe sur n'importe quel poste avec un simple clone du dépôt suivi de « docker compose up », dans des conditions proches de la production.

2. Précisez les moyens utilisés :

VS Code, Node.js 22, NestJS CLI, Vite, Git / GitHub, Docker et Docker Compose, MySQL 8.4, Redis, MinIO, Mailpit, ESLint, Prettier, OpenSpec

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce projet seul, dans le cadre du projet chef-d'œuvre de ma formation.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprint 0 : cadrage et initialisation)*

Période d'exercice ▶ Du : **04/08/26** au : **08/08/26**

5. Informations complémentaires (facultatif)

L'environnement est entièrement reproductible grâce à Docker Compose ; les mêmes images servent ensuite de base au déploiement en production.

Activité-type 1 Développer une application sécurisée

Exemple n°2 ▶ Interface React responsive avec navigation et API externe

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du brief « Interface React responsive avec navigation et API externe », nous devons développer en trois jours une application React dynamique qui consomme une API web externe, à partir des besoins d'un client fictif.

- Traduction des besoins du client en diagramme UML de cas d'utilisation, puis réalisation des maquettes et de l'enchaînement des écrans avant de coder.
- Création de l'application React avec une architecture en composants réutilisables et une arborescence de dossiers claire (pages, composants, services d'appel à l'API).
- Mise en place de la navigation entre les pages avec React Router : accueil, liste, page de détail et page d'erreur.
- Consommation de l'API externe avec fetch : gestion de l'asynchrone avec useEffect, stockage des données avec useState, affichage d'un état de chargement et d'un message en cas d'erreur.
- Intégration responsive (mobile, tablette, ordinateur) en soignant l'ergonomie et la lisibilité.
- Versionnement régulier avec Git, rédaction d'une documentation technique bilingue (français / anglais) et présentation de l'application lors d'une réunion client simulée.

2. Précisez les moyens utilisés :

React, React Router, JavaScript, Fetch API, CSS responsive, outil de maquettage, diagramme UML de cas d'utilisation, Git / GitHub

3. Avec qui avez-vous travaillé ?

En équipe de 3 à 4 apprenants de ma promotion : nous nous sommes réparti les écrans à développer et avons regroupé notre travail avec Git.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service ▶ *Formation CDA – Brief « Interface React responsive avec navigation et API externe »*

Période d'exercice ▶ Du : **20/01/26** au : **22/01/26**

5. Informations complémentaires (facultatif)

J'ai réinvesti ces compétences dans PromptVerse, dont j'ai développé toute l'interface en React 19 et TypeScript : catalogue, fiche d'un prompt, formulaires d'authentification, tableau de bord acheteur / vendeur et espace d'administration, avec React Router, TanStack Query, Zustand et Tailwind CSS.

Activité-type 1 Développer une application sécurisée

Exemple n°3 ▶ Composants métier de publication et d'achat de prompts – PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans PromptVerse, tout utilisateur inscrit peut publier des prompts et en acheter. J'ai développé dans l'API NestJS les composants métier qui portent ces règles, en les isolant dans des services (PromptsService, PurchasesService) appelés par les contrôleurs.

- Publication d'un prompt : validation des données entrantes par des DTO (class-validator), vérification de la catégorie et de l'outil IA choisis, génération d'un slug unique et mise en ligne immédiate.
- Protection du contenu payant : le texte du prompt n'est jamais renvoyé dans les listes, et sur la fiche il n'est visible que par le vendeur, par un acheteur dont l'achat est confirmé, ou par tous si le prompt est gratuit (prix à 0 €).
- Achat avec Stripe Checkout : avant de créer la session de paiement, le service applique les règles métier (impossible d'acheter son propre prompt, un prompt déjà acheté ou un prompt gratuit) et renvoie un code d'erreur explicite. Une tentative d'achat en attente est réutilisée au lieu de créer une nouvelle ligne à chaque clic.
- Traitement du webhook Stripe : vérification de la signature, passage de l'achat à l'état « COMPLETED », incrément du nombre de ventes et crédit du solde du vendeur. Le traitement est idempotent : si Stripe renvoie le même événement, le vendeur n'est pas crédité deux fois.
- Sécurité : authentification JWT transmise dans un cookie httpOnly, mots de passe hachés avec bcrypt, guard de rôles (USER, ADMIN, SUPER_ADMIN), limitation du nombre de requêtes sur les routes sensibles et en-têtes de sécurité avec Helmet.

2. Précisez les moyens utilisés :

NestJS 11, TypeScript, TypeORM, MySQL, Stripe Checkout et webhooks, class-validator, JWT, bcrypt, Helmet, @nestjs/throttler, Swagger, Jest

3. Avec qui avez-vous travaillé ?

J'ai travaillé seul, en m'appuyant sur la documentation officielle de NestJS et de Stripe ; les points bloquants étaient discutés avec mon formateur.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprints 2 et 3 : cœur métier, sécurité)*

Période d'exercice Du : **10/08/26** au : **11/09/26**

5. Informations complémentaires (facultatif)

Les règles métier de l'achat sont couvertes par des tests unitaires Jest (voir activité-type 3, exemple n°1).

Activité-type 1 Développer une application sécurisée

Exemple n°4 ► Organisation d'un projet full-stack en binôme (React, API REST, MySQL)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le brief « Application web full-stack avec React, API REST et MySQL » demandait de livrer en six jours, en binôme, une application complète. Le planning imposait un découpage en trois phases : conception de la base de données (jours 1 et 2), back-end (jours 3 et 4), puis front-end (jours 5 et 6). J'ai contribué à l'organisation du projet autant qu'au développement.

- Analyse du besoin, rédaction des user stories et priorisation des fonctionnalités pour tenir le délai.
- Découpage de chaque phase en tâches, suivi de leur avancement dans un tableau (à faire / en cours / terminé) et répartition des tâches avec mon binôme.
- Point d'avancement quotidien pour identifier les blocages et réajuster le planning.
- Organisation du dépôt Git commun : branches par fonctionnalité, commits réguliers et fusion après relecture.
- Tests des routes de l'API avec Postman avant de brancher le front-end, pour sécuriser l'intégration.
- Rédaction de la documentation technique et préparation de la démonstration finale.

2. Précisez les moyens utilisés :

Tableau de suivi des tâches (Kanban), Git / GitHub, Postman, React, Node.js / Express, MySQL

3. Avec qui avez-vous travaillé ?

En binôme avec un autre apprenant de ma promotion.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Brief « Application web full-stack avec React, API REST et MySQL »*

Période d'exercice Du : **05/02/26** au : **12/02/26**

5. Informations complémentaires (facultatif)

J'ai appliqué la même démarche à plus grande échelle sur PromptVerse, en méthode Agile Scrum : backlog priorisé sur Trello, huit sprints documentés du cadrage à la livraison et spécifications OpenSpec rédigées avant chaque développement.

Activité-type 2

Concevoir et développer une application sécurisée organisée en couches

Exemple n°1 ► *Analyse du besoin, UML et maquettes d'une application de gestion de projets*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le brief « Conception d'une application web : UML, maquettes, API REST sécurisée et interface React » consistait à concevoir et développer seul, en six jours, une application de gestion de projets et de tâches. La première partie du travail portait sur l'analyse du besoin et la conception.

- Étude du besoin et identification des acteurs et des fonctionnalités attendues : authentification, gestion des projets, gestion des tâches associées.
- Rédaction des user stories et des règles de gestion.
- Réalisation du diagramme de cas d'utilisation, du diagramme de classes et d'un diagramme de séquence, en vérifiant leur cohérence entre eux.
- Conception des maquettes des écrans principaux (connexion, liste des projets, détail d'un projet et de ses tâches) et de l'enchaînement entre les écrans.
- Conception de la base MySQL à partir du diagramme de classes, puis développement de l'API REST avec authentification JWT et d'une interface React fidèle aux maquettes.

Cette phase de conception m'a permis de valider le périmètre avant de coder et d'éviter des retours en arrière pendant le développement.

2. Précisez les moyens utilisés :

Outil de modélisation UML, outil de maquettage, MySQL, Node.js / Express, JWT, React, Git / GitHub

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce brief seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service ► *Formation CDA – Brief « Conception d'une application web : UML, maquettes, API REST sécurisée et interface React »*

Période d'exercice ► Du : **09/03/26** au : **16/03/26**

5. Informations complémentaires (facultatif)

Pour PromptVerse, j'ai suivi la même démarche : cahier des charges, cas d'utilisation, user stories avec critères d'acceptation et maquettes UX/UI en dark mode, réalisés avant le développement.

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

Exemple n°2 ▶ Architecture en couches de PromptVerse (NestJS / React)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Lors du sprint de conception de PromptVerse, j'ai transformé les besoins identifiés au cadrage en une architecture technique documentée avant de développer.

- Choix d'une architecture découplée : une API REST NestJS consommée par une application monopage React, chacune construite et déployée dans son propre conteneur.
- Côté API, organisation en modules métier (auth, users, prompts, categories, ai-tools, purchases, admin, storage, mail) et en couches : contrôleurs pour les routes HTTP, services pour les règles métier, repositories TypeORM pour l'accès aux données et DTO pour la validation des entrées. Les éléments transverses (guards, décorateurs, codes d'erreur, validation de la configuration) sont regroupés dans un dossier commun.
- Côté client, séparation des pages, des composants d'interface, des hooks d'accès aux données (TanStack Query) et des stores d'état client (Zustand), avec un client HTTP centralisé (Axios).
- Modélisation UML de l'architecture : diagramme de composants, diagramme de classes et diagrammes de séquence de l'authentification, de la création d'un prompt et de l'achat.
- Intégration des services externes : Stripe pour le paiement, MinIO (S3) pour les images, Redis pour le cache et un serveur SMTP pour les emails.
- Documentation de l'API avec Swagger / OpenAPI et traçabilité des décisions dans les propositions OpenSpec.

2. Précisez les moyens utilisés :

UML (diagrammes Mermaid), NestJS, TypeORM, React, TanStack Query, Zustand, Axios, Swagger / OpenAPI, OpenSpec, Git / GitHub

3. Avec qui avez-vous travaillé ?

J'ai conçu cette architecture seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

►

Chantier, atelier, service ► *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprint 1 : conception et architecture)*

Période d'exercice ► Du : **04/08/26** au : **08/08/26**

5. Informations complémentaires (facultatif)

Les diagrammes UML et les choix d'architecture sont versionnés dans le dépôt, avec le code, dans le dossier docs/.

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

Exemple n°3 ▶ Modélisation Merise et base MySQL de PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai conçu la base de données relationnelle de PromptVerse avec la méthode Merise, puis je l'ai mise en place dans MySQL à l'aide de migrations TypeORM.

- Rédaction du dictionnaire de données et des règles de gestion : un utilisateur possède un profil et publie des prompts ; un prompt appartient à une catégorie et à un outil IA ; un achat relie un acheteur, un vendeur et un prompt.
- Réalisation du MCD, puis passage au MLD et au MPD (script SQL MySQL) : tables users, user_profiles, social_links, categories, ai_tools, prompts, preview_images et purchases.
- Définition des contraintes d'intégrité : identifiants UUID, clés étrangères, unicité de l'email, du nom d'utilisateur et des slugs, index sur les colonnes filtrées ou triées (statut, date de création, nombre de ventes).
- Choix des règles de suppression : suppression en cascade des données dépendantes (profil, liens sociaux, images d'aperçu) et restriction sur les prompts et les achats pour ne jamais perdre l'historique des ventes.
- Stockage des montants en DECIMAL(10,2) pour éviter les erreurs d'arrondi.
- Traduction du modèle en entités TypeORM et création de migrations versionnées, exécutées automatiquement au démarrage du conteneur de l'API.

2. Précisez les moyens utilisés :

Méthode Merise (dictionnaire de données, MCD, MLD, MPD), MySQL 8.4, TypeORM (entités et migrations), Docker

3. Avec qui avez-vous travaillé ?

J'ai réalisé cette modélisation seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprint 1 : conception de la base de données)*

Période d'exercice Du : **05/08/26** au : **10/08/26**

5. Informations complémentaires (facultatif)

Pendant la formation, j'avais déjà conçu un MCD / MLD et une base MySQL en binôme lors du brief « Application web full-stack avec React, API REST et MySQL ».

Activité-type 2 Concevoir et développer une application sécurisée organisée en couches

Exemple n°4 ▶ Accès aux données MySQL (TypeORM) et Redis – PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans l'API de PromptVerse, j'ai développé les composants qui lisent et écrivent les données, en SQL avec MySQL et en NoSQL avec Redis.

- Accès SQL par des repositories TypeORM injectés dans les services : catalogue paginé et trié par date, listes filtrées par catégorie et par outil IA, limitées aux prompts publiés.
- Sélection explicite des colonnes renvoyées dans les listes : le contenu payant n'est jamais chargé, ce qui allège les réponses et évite toute fuite de données.
- Requêtes paramétrées générées par l'ORM, qui protègent contre les injections SQL, et validation des paramètres reçus par des DTO.
- Écritures fiables : incrément atomique du compteur de ventes, mise à jour conditionnelle d'un achat (passage à « FAILED » uniquement s'il est encore en attente).
- Accès NoSQL avec Redis, base clé-valeur utilisée comme cache applicatif via le module de cache de NestJS (durée de vie de 10 minutes), avec invalidation des clés concernées lorsque les données changent, par exemple après une inscription.
- Stockage objet compatible S3 (MinIO) pour les images de couverture et d'aperçu, dont les références sont enregistrées en base.

2. Précisez les moyens utilisés :

TypeORM, MySQL 8.4, Redis, @nestjs/cache-manager et Keyv, MinIO (S3), class-validator, Swagger

3. Avec qui avez-vous travaillé ?

J'ai réalisé ces développements seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

►

Chantier, atelier, service ► *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprints 2 à 4 : développement et performance)*

Période d'exercice ► Du : **08/08/26** au : **11/09/26**

5. Informations complémentaires (facultatif)

Activité-type 3 **Préparer le déploiement d'une application sécurisée**

Exemple n°1 ▶ Plan de tests et tests unitaires Jest – PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour sécuriser les fonctionnalités sensibles de PromptVerse, j'ai préparé un plan de tests à partir des critères d'acceptation rédigés dans OpenSpec, puis je l'ai exécuté.

- Pour chaque règle métier, identification du cas nominal et des cas d'erreur attendus.
- Écriture de tests unitaires avec Jest en isolant la base de données et Stripe par des mocks :
 - PromptsService (6 cas) : visibilité du contenu payant pour un visiteur anonyme, un utilisateur non acheteur, le vendeur, un acheteur confirmé et pour un prompt gratuit ; prompt inexistant.
 - PurchasesService (9 cas) : refus d'acheter son propre prompt, un prompt gratuit ou déjà acheté ; création puis réutilisation d'un achat en attente ; signature de webhook invalide ; crédit du vendeur ; idempotence ; événements non gérés.
 - AdminUserService (3 cas) : changement de rôle d'un utilisateur, utilisateur inexistant, rôle invalide.
- Exécution des tests, mesure de la couverture de code et correction des anomalies détectées.
- Tests manuels des routes avec Swagger et des parcours complets (inscription, publication, achat en mode test Stripe).
- Intégration du lint et des tests dans le pipeline GitHub Actions : un échec bloque la construction des images et le déploiement.

2. Précisez les moyens utilisés :

Jest, ts-jest, mocks des repositories et de Stripe, couverture de code, Swagger, mode test Stripe, GitHub Actions

3. Avec qui avez-vous travaillé ?

J'ai réalisé ces tests seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprint 3 : sécurité, qualité et tests)*

Période d'exercice Du : **09/09/26** au : **11/09/26**

5. Informations complémentaires (facultatif)

Pendant la formation, j'ai également travaillé sur les tests unitaires, les tests d'intégration de l'API et les tests End-to-End lors des briefs dédiés à la qualité logicielle.

Activité-type 3 **Préparer le déploiement d'une application sécurisée**

Exemple n°2 ▶ Dockeriser une application et documenter son déploiement

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le brief « Dockeriser une application existante et mettre en place un pipeline de CI » avait pour but de rendre une application reproductible et facile à déployer, et de documenter toute la démarche.

- Écriture d'un Dockerfile multi-stage : une étape de build (installation des dépendances et compilation) et une image finale légère qui ne contient que le nécessaire à l'exécution.
- Gestion de la configuration par variables d'environnement, sans secret dans l'image.
- Écriture d'un docker-compose.yml pour lancer l'application et ses services (base de données).
- Mise en place d'un pipeline de CI avec GitHub Actions : installation des dépendances, lint, tests automatisés et construction de l'image Docker à chaque push.
- Rédaction d'un README complet : prérequis, installation, lancement en local et avec Docker, variables d'environnement, fonctionnement du pipeline et exécution des tests.

2. Précisez les moyens utilisés :

Docker, Docker Compose, GitHub Actions, Git / GitHub, Node.js, Markdown

3. Avec qui avez-vous travaillé ?

En binôme avec un autre apprenant de ma promotion.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Brief « Dockeriser une application existante et mettre en place un pipeline de CI »*

Période d'exercice Du : **04/05/26** au : **07/05/26**

5. Informations complémentaires (facultatif)

J'ai appliqué la même démarche sur PromptVerse : Dockerfiles multi-stage pour l'API et le client, fichiers Compose distincts pour le développement et la production, et pipeline CI/CD complet (voir exemple n°3).

Activité-type 3 **Préparer le déploiement d'une application sécurisée**

Exemple n°3 ▶ Chaîne CI/CD et mise en production de PromptVerse

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

J'ai mis PromptVerse en production sur un serveur VPS, avec une chaîne d'intégration et de déploiement continu. L'application est accessible en ligne à l'adresse <https://promptverse.biz>.

- Dockerfiles multi-stage pour l'API (compilation TypeScript puis image d'exécution) et pour le client (build Vite servi par Nginx).
- Script de démarrage du conteneur de l'API : attente de la disponibilité de MySQL, exécution automatique des migrations, puis lancement de l'application.
- Fichier docker-compose.prod.yml sur le serveur : Nginx en frontal, API, client, MySQL, Redis et MinIO, avec des volumes persistants pour les données.
- Configuration de Nginx en reverse proxy : routage vers l'API et le client, HTTPS avec un certificat Let's Encrypt obtenu avec Certbot, limite de taille des fichiers envoyés.
- Pipeline GitHub Actions déclenché sur la branche de déploiement : tests et lint du back-end avec une base MySQL de test, lint et build du front-end, construction et publication des images sur Docker Hub, puis déploiement sur le VPS par SSH.
- Secrets de production (accès Docker Hub et SSH, clés Stripe, secret JWT) stockés dans les secrets GitHub et sur le serveur.
- Route de santé (health check) pour vérifier que l'API répond après chaque déploiement.

2. Précisez les moyens utilisés :

Docker, Docker Hub, Docker Compose, GitHub Actions, Nginx, Let's Encrypt / Certbot, VPS Linux, SSH

3. Avec qui avez-vous travaillé ?

J'ai réalisé cette mise en production seul.

4. Contexte

Nom de l'entreprise, organisme ou association *Simplon*

Chantier, atelier, service *Formation CDA – Projet chef-d'œuvre PromptVerse (Sprints 5 et 6 : industrialisation et mise en production)*

Période d'exercice Du : **10/09/26** au : **14/09/26**

5. Informations complémentaires (facultatif)

Chaque push sur la branche de déploiement déclenche automatiquement les tests, la construction des images et la mise à jour du serveur.

DOSSIER PROFESSIONNEL ^(DP)

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Cliquez ici.	Cliquez ici pour taper du texte.	Cliquez ici pour sélectionner une date.

Déclaration sur l'honneur

Je soussigné Abduljaber AL OMRAN ,
déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et
que je suis l'auteur(e) des réalisations jointes.

Fait à Thiers, le
pour faire valoir ce que de droit.

Signature :

Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Dépôt GitHub du projet PromptVerse et application en production : https://promptverse.biz

ANNEXES

(Si le RC le prévoit)